

Detecção de Texto Gerado por IA: Deep Learning e Few-Shot Prompting com LLMs

Luís Silva, Pedro Reis, Guilherme Pinto e Pedro Gomes

Mestrado em Inteligência Artificial, Universidade do Minho, Braga, Portugal
{pg60390, pg59908, pg60225, pg60289}@alunos.uminho.pt

Abstract. A detecção automática de texto gerado por IA é um problema crescente com implicações na integridade académica e na moderação de conteúdo. Neste trabalho, abordamos uma formulação multi-classe — classificar textos curtos (80–120 palavras) em cinco categorias (Human, Anthropic, Google, Meta, OpenAI) — através de três abordagens complementares: análise e curadoria dos dados de treino para mitigar a mudança de domínio, modelos de *Deep Learning* treinados localmente, e LLMs como classificadores via *few-shot prompting* [1]. Nos dados do docente, o melhor modelo treinado localmente atinge 77,26%, enquanto o Claude Opus 4.6 atinge 87,33% sem treino específico, e um meta-classificador por *stacking* sobre três LLMs atinge 92,67%. Apesar de os modelos treinados atingirem >94% em validação cruzada interna, nenhum ultrapassa 78% no teste externo, evidenciando sensibilidade à mudança de domínio, enquanto os LLMs demonstram maior robustez.

Keywords: Texto gerado por IA · Classificação multi-classe · *Deep Learning* · *Few-shot prompting* · *Large Language Models*

1 Introdução

Com o aparecimento de vários modelos de linguagem, a qualidade do texto gerado por IA aproxima-se cada vez mais da escrita humana. Distinguir automaticamente entre texto humano e texto gerado por diferentes modelos tornou-se relevante em áreas como a integridade académica e a moderação de conteúdo.

O presente trabalho aborda este problema numa formulação multi-classe com cinco categorias: textos escritos por humanos e textos gerados por quatro famílias de modelos — Anthropic, Google, Meta e OpenAI. Os textos são curtos (80–120 palavras) e centrados em ciências naturais e tecnologia. O enunciado original incluía a classe Mistral, substituída pela Anthropic. Todo o código, notebooks e documentação estão disponíveis no repositório do grupo¹.

Explorámos três estratégias complementares: análise e curadoria dos dados de treino para mitigar a mudança de domínio (Secção 2), incluindo seleção combinatória de 180 datasets e extração de métricas estilísticas; modelos de Deep Learning treinados localmente (Secções 3 e 4), desde DNNs implementadas de

¹ <https://github.com/Luismpos/AP>

raiz até fine-tuning de Transformers; e LLMs com few-shot prompting (Secção 5), usando Claude Opus, Gemini 3.1 Pro, GPT-5.4 e DeepSeek V3.

A análise exploratória dos dados — incluindo a extração de 24 métricas comportamentais e a seleção combinatória de 180 combinações de modelos — foi desenvolvida especificamente para aproximar a distribuição de treino à do docente e mitigar a mudança de domínio. Apesar destas medidas, os modelos treinados atingem no máximo $\sim 77\%$ nos dados do docente, face a $>94\%$ em validação cruzada. A introdução de LLMs como classificadores diretos contorna este problema: o Claude Opus atinge $\sim 87\%$ sem treino dedicado, e um meta-classificador por stacking sobre três LLMs atinge $\sim 93\%$.

2 Preparação e Curadoria dos Dados

A qualidade dos dados de treino é determinante para o desempenho dos modelos. Descrevemos o processo de compilação e seleção, incluindo a estratégia combinatória para aproximar a distribuição de treino à dos dados do docente.

2.1 Fontes e Geração de Dados

Numa fase exploratória, testámos datasets públicos do HuggingFace (OpenTuringBench, HC3, M4). Contudo, a diferença de domínio face aos 125 exemplos do docente era demasiado acentuada, pelo que optámos por geração própria via APIs de 15 modelos: GPT-3.5-Turbo, GPT-4o, GPT-4o-mini e GPT-5o-mini (OpenAI); Gemini 2.5 Pro, Gemini Flash, Gemma 1/2/3 (Google); Opus 4.6, Sonnet 4.6 e Haiku 4.5 (Anthropic); e Llama 3, 3.1 e 3.2 (Meta). Os prompts foram orientados para temas de ciências naturais com 80–120 palavras.

Os textos humanos foram extraídos de revisões da Wikipedia anteriores a 2021 via API, a partir de $\sim 24\,000$ termos científicos, garantindo anterioridade aos modelos atuais. Complementámos com geração few-shot [1]: os textos do docente serviram como exemplos de estilo, guiando cada modelo a imitar as características dos dados de avaliação. Os scripts de geração e expansão do dataset encontram-se em `database/func/` no repositório (ficheiros `data.py`, `extend.py` e `fewshot.py`).

2.2 Seleção Combinatória e Pré-processamento

Como o docente não especificou as versões dos modelos usadas, gerámos o produto cartesiano de todas as combinações de ficheiros por categoria, resultando em 180 datasets candidatos. Cada combinação foi avaliada com regressão logística (*baseline*) + TF-IDF contra os 125 exemplos do docente (ver `notebooks/Data.ipynb`). A combinação ótima foi `human.csv + gpt-4o.csv + gemma3.csv + sonnet4.6.csv + llama3.2.csv`.

O pré-processamento incluiu limpeza de artefactos (referências, IPA, formatação), remoção de duplicados, truncagem em fronteiras de frase (máximo 120 palavras), e balanceamento por sub-amostragem. O dataset final contém

~122 000 textos equilibrados pelas cinco classes. Para validação interna, usámos Stratified K-Fold (K=3) com TF-IDF ajustado apenas nos dados de treino de cada fold. O dataset de teste externo combina 475 textos do docente.

3 Modelos com Implementação Própria

Para a Tarefa 2 do enunciado, implementámos de raiz uma framework modular de Deep Learning em NumPy, organizada em módulos independentes no diretório `src/`: camadas (`layers.py`), ativações (`activations.py`), otimizadores (`optimizer.py`), funções de custo (`losses.py`), rede neuronal (`neuralnet.py`) e vectorizadores (`vectorizers.py`).

3.1 Arquitetura da Framework

A camada `DenseLayer` realiza $\mathbf{z} = \mathbf{X}\mathbf{W} + \mathbf{b}$ no forward pass e propaga $\delta\mathbf{W}^\top$ no backward pass, com gradientes $\partial L/\partial \mathbf{W} = \mathbf{X}^\top \delta$. Os pesos usam He initialization e a camada suporta regularização L2. A camada `DropoutLayer` implementa *inverted dropout*, escalando por $1/(1-p)$ durante o treino. As ativações incluem ReLU e Softmax com estabilidade numérica; a combinação Softmax + Cross-Entropy simplifica a derivada conjunta para $\hat{\mathbf{y}} - \mathbf{y}$.

Implementámos SGD com Momentum ($\mathbf{v}_t = \mu\mathbf{v}_{t-1} + \mathbf{g}_t$) e Adam [5] com bias correction. O ciclo de treino usa mini-batches com *early stopping*: a loss de validação é monitorizada e, sem melhoria durante p épocas, o treino é interrompido e os pesos da melhor época restaurados.

3.2 Representação dos Textos e Resultados

A conversão para representação tabular foi inteiramente implementada em NumPy. O vectorizador TF-IDF calcula $\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$, com TF sublinear ($1 + \log(\text{tf})$) e IDF suavizado ($\log \frac{1+n}{1+\text{idf}(t)} + 1$), seguido de normalização L2 por documento. Gerámos n-gramas de palavras (1–2) e n-gramas de caracteres com word boundaries (2–4), totalizando 5013 features (3000 word n-gramas + 2000 character n-gramas + 13 estilísticas). O vectorizador suporta filtragem por frequência mínima e máxima de documentos, e inclui uma lista de stop words para inglês.

As 5000 features TF-IDF são complementadas por 13 features estilísticas extraídas do texto original: comprimento em caracteres e palavras, comprimento médio e desvio padrão de palavras e frases, riqueza vocabular (Type-Token Ratio), número de frases, proporção de pontuação, dígitos e maiúsculas, e densidade de vírgulas e pontos finais. A normalização é feita via `StandardScaler` e `OneHotEncoder`, ambos implementados de raiz. A função de custo é a Categorical Cross-Entropy com clipping ($\epsilon = 10^{-15}$) para estabilidade numérica.

Numa análise exploratória separada (`notebooks/Data.ipynb`), extraímos adicionalmente 24 métricas comportamentais avançadas — incluindo Burstiness,

Perplexity, POS Entropy e GLTR [3] — que revelaram padrões estilísticos distintos entre classes. A visualização sob a forma de heatmap de “impressão digital” mostrou, por exemplo, que os textos Human apresentam maior variância no comprimento de frases e menor Buzzword Ratio do que os textos gerados por IA.

Como baseline, implementámos Regressão Logística Multi-classe (módulo `logisticregression.py`) com regularização L_2^2 . Testámos cinco configurações de DNN em K-Fold ($K=3$). Todas incluem Dropout (0,3 por defeito), variando largura, taxa de Dropout e regularização L_2 (Tabela 1). Os resultados completos, incluindo curvas de aprendizagem e confusion matrices, encontram-se no notebook `Numpy.ipynb`.

Table 1. Resultados dos modelos NumPy — validação cruzada e teste do docente (475 textos). Todas as DNNs usam o otimizador Adam ($lr=10^{-4}$).

Modelo	K-Fold CV	Teste
Regressão Logística (baseline)	90,57%	69,47%
DNN 128→64 (dropout=0,3 e $L_2=0$)	94,49%	75,37%
DNN 256→128 (dropout=0,3 e $L_2=0$)	94,55%	74,11%
DNN 128→128 (dropout=0,3 e $L_2=0$)	94,54%	73,47%
DNN 256→128 (dropout=0,5 e $L_2=0$)	94,59%	74,74%
DNN 256→128 (dropout=0,5 e $L_2=0,01$)	93,88%	73,26%

A queda de ~ 20 pontos percentuais entre CV e teste evidencia que, apesar da seleção combinatória de dados, a mudança de domínio persiste. O melhor modelo em CV não coincide com o melhor no teste, reforçando que a validação interna não é preditor fiável da generalização neste cenário.

4 Modelos em PyTorch

Em PyTorch, explorámos famílias de modelos com complexidade crescente, desde DNNs tabulares até Transformers pré-treinados, com treino em GPU (NVIDIA RTX com CUDA) e comparação via K-Fold ($K=3$) e Grid Search. Os resultados detalhados encontram-se nos notebooks `notebooks/Pytorch.ipynb` (DNNs e modelos sequenciais) e `notebooks/Transformers.ipynb` (BERT, DistilBERT, RoBERTa).

4.1 DNNs Tabulares e Modelos Sequenciais

Melhorámos a abordagem NumPy com BatchNorm e ativações alternativas (LeakyReLU, ELU), testando sete variantes de DNN que exploram largura (128

² Na Regressão Logística, o gradiente de regularização é normalizado pelo tamanho do mini-batch ($\nabla_{\mathbf{W}} \mathcal{R} = \frac{\lambda}{m} \mathbf{W}$), enquanto nas DNNs é aplicado diretamente ($\nabla_{\mathbf{W}} \mathcal{R} = \lambda \mathbf{W}$), seguindo convenções distintas mas ambas válidas.

a 256 neurónios), profundidade (2 a 4 camadas) e função de ativação. A introdução de BatchNorm estabiliza o treino ao normalizar as ativações intermédias. O Grid Search sobre a arquitetura DNNGrid explorou 27 combinações de otimizador (Adam, AdamW, SGD), learning rate ($\{10^{-2}, 10^{-3}, 10^{-4}\}$) e Dropout ($\{0, 3; 0, 5; 0, 7\}$), obtendo 95,00% em CV com Adam, lr= 10^{-3} e Dropout 0,5.

Para os modelos sequenciais, tokenizámos os textos num vocabulário de 30 000 palavras (comprimento máximo 150 tokens). Treinámos embeddings de palavras com masked mean pooling alimentando uma DNN — a máscara ignora os tokens de padding no cálculo da média, garantindo que a representação reflete apenas os tokens reais. O Embedding 256d atingiu 76,00% no teste do docente, superando várias DNNs tabulares, o que sugere que representações semânticas mais simples podem generalizar melhor quando existe mudança de domínio.

Os modelos recorrentes operam diretamente sobre as sequências de tokens. O BiLSTM [4] bidirecional com duas camadas concatena os estados ocultos das duas direções no último passo temporal e alimenta um classificador denso com duas camadas (128→64) e Dropout. O BiGRU, com menos parâmetros (sem *cell state*), obteve resultados competitivos: o BiGRU (h=256) atingiu 76,84% no teste do docente, o melhor entre não-Transformers. Testámos também BiLSTM com GloVe [7] pré-treinado (100d, 400k vetores, 90,6% de cobertura do vocabulário) nos modos *frozen* e *fine-tuned*, observando que o fine-tuning melhora ligeiramente o desempenho (75,79% vs. 74,74% no teste).

4.2 Transformers Pré-treinados

Realizámos fine-tuning de três modelos encoder-only do HuggingFace: BERT [2] (110M parâmetros), DistilBERT [8] (66M, 40% menos parâmetros com 97% do desempenho original) e RoBERTa [6] (125M, treinado com mais dados e sem *next sentence prediction*). O classificador consiste numa camada linear com Dropout sobre a representação do token [CLS], que captura informação global da sequência. Os textos são tokenizados com o tokenizer respetivo de cada modelo (não intercambiáveis entre famílias) com comprimento máximo de 128 subtokens — adequado para textos de 80–120 palavras, que produzem tipicamente 100–130 subtokens.

Comparámos duas estratégias de congelamento: *frozen* (apenas o classification head treinável, com learning rate 10^{-3}) e *partial* (últimas N camadas do encoder descongeladas, com learning rate 2×10^{-5}). A diferença foi substancial: os modelos *frozen* atingiram apenas 64–68% em K-Fold e 50–53% no teste do docente, enquanto os *partial* ultrapassaram 91% em CV, confirmando que as representações pré-treinadas necessitam de adaptação à tarefa.

O Grid Search sobre DistilBERT explorou 27 configurações (learning rate $\in \{10^{-5}, 2 \times 10^{-5}, 5 \times 10^{-5}\}$, Dropout $\in \{0, 2; 0, 3; 0, 5\}$, camadas descongeladas $\in \{1, 2, 4\}$), atingindo 96,15% em CV com lr= 5×10^{-5} , Dropout 0,2 e 4 camadas descongeladas. O treino utilizou mixed precision (FP16) para reduzir o consumo de VRAM, linear warmup scheduler sobre 10% dos passos totais para estabilizar o início do treino, e checkpoints por fold para recuperação em caso de falha.

4.3 Resultados Comparativos

A Tabela 2 resume os melhores resultados de cada família.

Table 2. Resultados dos modelos PyTorch — validação cruzada e teste do docente (475 textos).

Modelo	K-Fold CV	Teste
DNN Grid Search (Adam, lr= 10^{-3} , drop=0,5)	95,00%	75,16%
Embedding 256d + DNN	90,69%	76,00%
BiLSTM (h=256)	94,11%	73,68%
BiGRU (h=256)	94,72%	76,84%
BiLSTM + GloVe (fine-tuned)	93,40%	75,79%
BERT (partial)	91,11%	69,68%
DistilBERT (partial)	93,85%	74,32%
RoBERTa (partial)	93,13%	76,00%
DistilBERT Grid Search	96,15%	77,26%

O DistilBERT com Grid Search atinge simultaneamente o melhor CV (96,15%) e o melhor resultado no teste do docente (77,26%) entre os modelos treinados, superando ligeiramente o BiGRU (76,84%). De notar que o desempenho em CV não se traduz linearmente em melhor generalização — o Embedding 256d, com apenas 90,69% em CV, atinge 76,00% no teste, superando várias DNNs com >94% em CV. Mesmo o melhor modelo treinado (77,26%) fica aquém do desejável, motivando a exploração de LLMs na secção seguinte.

5 Classificação via Few-Shot Prompting com LLMs

Face à persistência da mudança de domínio nos modelos treinados — apesar da curadoria dos dados — explorámos LLMs como classificadores diretos [1], uma abordagem que dispensa dados de treino específicos. A classificação individual está no notebook `LLM.ipynb` e o ensemble com stacking no notebook `Ensemble.ipynb`.

5.1 Configuração e Prompting

Avaliámos quatro LLMs de fornecedores distintos: Claude Opus 4.6 [9], Gemini 3.1 Pro [10], GPT-5.4 [11] e DeepSeek V3 [12]. Todos foram chamados com temperatura 0 (resposta determinística) e `max_tokens=10`, dado que a resposta esperada é apenas o nome da classe.

Cada chamada inclui um prompt estruturado em três partes: uma instrução de sistema que define a tarefa e enumera as cinco classes com uma breve descrição de cada uma (e.g., “Human — written by a human, e.g. from Wikipedia”); um support set de exemplos rotulados, onde cada exemplo apresenta o texto seguido

da sua classe; e finalmente o texto a classificar, com a instrução de responder apenas com o nome da classe. O número de exemplos por classe variou entre 10 (validação) e 40 (submissões). A normalização da resposta é feita por correspondência parcial de strings (e.g., “anthropic” → Anthropic).

5.2 Ensemble e Stacking

Reservámos 10 exemplos por classe como support set e 150 textos como validação — nunca incluídos no prompt. Testámos três esquemas de votação: maioritária ($w_i = 1/N$), ponderação linear ($w_i \propto \text{acc}_i$) e quadrática ($w_i \propto \text{acc}_i^2$), onde o label final é:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \sum_{i=1}^N w_i \cdot \mathbb{I}[f_i(x) = c] \quad (1)$$

Contudo, estes esquemas tratam todos os textos uniformemente. Observando que cada LLM tem forças complementares — o Claude domina Google e Human ($F_1 \geq 0,98$), o GPT tem 96% de recall em OpenAI, e o Gemini 95% em Meta — implementámos um meta-classificador por stacking que aprende a rotear cada texto para o modelo mais adequado.

As previsões dos LLMs são codificadas como features para o classificador de segundo nível: cada previsão é representada em one-hot ($4 \times 5 = 20$ features), complementada pela fração de concordância entre modelos e pela entropia das votações (22 features no total). Avaliámos Regressão Logística e MLP (32→16 neurónios, L2=0,01) com Leave-One-Out Cross-Validation (LOO-CV) para evitar *data leakage* sobre os 150 textos. Testámos também a ablação sem DeepSeek (42,67%), que se revelou benéfica ao reduzir o ruído e o número de features (17 em vez de 22).

A análise de complementaridade justifica esta abordagem: o Claude erra em 19 dos 150 textos, dos quais o GPT corrige 12 (63%) e o Gemini 7 (37%); apenas 5 erros não são corrigidos por nenhum modelo. O oracle — a accuracy se fosse possível escolher sempre o modelo correto — atinge 96,67%, confirmando elevada complementaridade entre os LLMs avaliados.

5.3 Resultados

A Tabela 3 apresenta os resultados de todos os esquemas nos dados do docente (150 textos).

O stacking MLP sem DeepSeek atinge 92,67%, superando o Claude solo em 5,3 p.p. O ganho provém da capacidade do meta-classificador de aprender padrões de desacordo entre os LLMs: quando o Claude e o GPT discordam, o meta-classificador tende a favorecer o GPT em textos OpenAI e Anthropic, e o Claude em textos Google e Human. A melhoria é mais substancial nas classes onde a confusão entre modelos é mais acentuada — Anthropic (recall de 59% → 81%) e OpenAI — enquanto o Claude mantém $F_1 \geq 0,98$ em Human e Google, classes onde o stacking preserva o desempenho individual.

A exclusão do DeepSeek (42,67%) melhora o resultado ao remover ruído e reduzir o risco de *overfitting* (17 features em vez de 22 sobre apenas 150 textos).

Table 3. Accuracy dos LLMs e esquemas de ensemble nos dados do docente (150 textos).

Modelo / Esquema	Accuracy
Claude Opus 4.6	87,33%
Gemini 3.1 Pro	81,33%
GPT-5.4	77,33%
DeepSeek V3	42,67%
Ponderação linear	85,33%
Stacking LR (4 LLMs, LOO-CV)	91,33%
Stacking MLP (3 LLMs, LOO-CV)	92,67%
Oracle	96,67%

6 Resultados e Discussão

Nesta secção confrontamos as abordagens nos dados do docente, analisamos a discrepância entre validação interna e teste externo, e apresentamos os rankings das submissões.

6.1 Comparação Global

A Tabela 4 confronta os melhores modelos de cada abordagem, todos avaliados nos dados do docente.

Table 4. Comparação dos melhores modelos por abordagem nos dados do docente.

Abordagem	Modelo	K-Fold CV Teste (docente)	
NumPy	DNN 128→64	94,49%	75,37%
PyTorch	BiGRU (h=256)	94,72%	76,84%
Transformer	DistilBERT Grid Search	96,15%	77,26%
LLM (solo)	Claude Opus 4.6	—	87,33%
LLM (stacking)	MLP sobre 3 LLMs	—	92,67%

A progressão é clara nos dados do docente: $\sim 75\%$ (NumPy) $\rightarrow \sim 77\%$ (DistilBERT) $\rightarrow \sim 87\%$ (Claude solo) $\rightarrow \sim 93\%$ (stacking). A diferença de >15 p.p. entre o melhor modelo treinado e o stacking sustenta a conclusão de que a combinação de LLMs é a estratégia mais eficaz.

6.2 Discrepância Treino–Teste e Rankings

A queda consistente de ~ 19 p.p. entre a validação cruzada e os dados do docente aponta para três fatores: mudança de domínio (as versões dos modelos do docente

introduzem variações estilísticas que os nossos dados não capturam, apesar da seleção combinatória), sobre-ajustamento a sinais espúrios (amplificado pelo fine-tuning profundo dos Transformers), e robustez relativa das features (o TF-IDF, por capturar sinais lexicais mais genéricos, é ligeiramente mais robusto do que embeddings contextuais). A abordagem com LLMs contorna estes problemas ao dispensar dados de treino, recorrendo ao conhecimento pré-existente sobre estilos de escrita de diferentes modelos.

A Tabela 5 apresenta os resultados obtidos nas três submissões ao docente. A evolução reflete a mudança estratégica: na submissão 1, apenas modelos treinados localmente (DNN NumPy e PyTorch); na submissão 2, a introdução do Claude Opus produziu um salto de 71% para 91%, ultrapassando a barreira imposta pela mudança de domínio; na submissão 3, ambos os slots foram preenchidos por LLMs, consolidando a vantagem da abordagem few-shot. O aumento no número de exemplos no support set (de 30 para 40 por classe) e o uso de 288 textos de treino acumulados permitiram manter a competitividade apesar do aumento progressivo na dificuldade dos textos de avaliação.

Table 5. Rankings das submissões (23–25 grupos).

Subm.	Modelos (A / B)	Acc. A	Acc. B	Ranking
1	DNN NumPy / DNN PyTorch	71,33%	68,67%	1. ^o / 25
2	Claude Opus (N=30) / DNN PyTorch	91,33%	72,67%	1. ^o / 24
3	Claude Opus (N=40) / Gemini Pro (N=20)	88,00%	83,33%	1. ^o / 24

Os notebooks de cada submissão encontram-se nas pastas `Subm1/`, `Subm2/` e `Subm3/` do repositório, incluindo o código para carregar os modelos treinados e gerar os ficheiros CSV de previsões.

7 Conclusões

Desenvolvemos e comparámos três famílias de modelos para deteção de texto gerado por IA.

Apesar do esforço na curadoria dos dados — incluindo a seleção combinatória de 180 datasets e a geração few-shot — os modelos treinados atingem no máximo $\sim 77\%$ nos dados do docente, face a $>94\%$ em validação cruzada. Os LLMs contornam esta mudança de domínio: o Claude Opus atinge $87,33\%$ sem treino específico, e o stacking sobre três LLMs ($92,67\%$) é a melhor abordagem, explorando a complementaridade entre modelos — o GPT corrige 63% dos erros do Claude.

Em suma, para problemas com distribuição de teste desconhecida e textos curtos, a combinação de múltiplos LLMs com um meta-classificador supervisionado oferece o melhor compromisso entre robustez e desempenho.

Este trabalho apresenta algumas limitações. O stacking é avaliado sobre apenas 150 textos, uma amostra que pode não capturar toda a variabilidade dos

dados. O uso do Claude para classificar textos que incluem a classe Anthropic introduz um potencial viés, já que o modelo pode reconhecer padrões do seu próprio estilo de escrita. Como trabalho futuro, seria interessante explorar técnicas de adaptação de domínio para reduzir a discrepância treino–teste, avaliar o impacto da calibração de confiança nas previsões antes do stacking, e investigar se LLMs *open-source* com acesso aos logits permitiriam melhorar a complementaridade do meta-classificador.

References

1. Brown, T., Mann, B., Ryder, N., et al.: Language models are few-shot learners. In: NeurIPS (2020). <https://arxiv.org/abs/2005.14165>
2. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: NAACL-HLT. pp. 4171–4186 (2019). <https://doi.org/10.18653/v1/N19-1423>
3. Gehrmann, S., Strobelt, H., Rush, A.M.: GLTR: Statistical detection and visualization of generated text. In: ACL (System Demonstrations). pp. 111–116 (2019). <https://doi.org/10.18653/v1/P19-3019>
4. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997). <https://doi.org/10.1162/neco.1997.9.8.1735>
5. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: ICLR (2015). <https://arxiv.org/abs/1412.6980>
6. Liu, Y., Ott, M., Goyal, N., et al.: RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692 (2019). https://doi.org/10.1007/978-3-030-84186-7_31
7. Pennington, J., Socher, R., Manning, C.D.: GloVe: Global vectors for word representation. In: EMNLP. pp. 1532–1543 (2014). <https://nlp.stanford.edu/projects/glove/>
8. Sanh, V., Debut, L., Chaumond, J., Wolf, T.: DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. In: EMC² Workshop, NeurIPS (2019). <https://arxiv.org/abs/1910.01108>
9. Anthropic: Claude API documentation (2026). <https://docs.anthropic.com>
10. Google: Gemini API documentation (2026). <https://ai.google.dev/gemini-api/docs>
11. OpenAI: API documentation (2026). <https://platform.openai.com/docs>
12. DeepSeek-AI: API documentation (2026). <https://api-docs.deepseek.com>